FILE PATH TRAVERSAL - SIMPLE CASE

TryHackMe Learning & Practical Report

Prepared by	Belal Rabea Khalifa
Program	IqraSec Academy, WAPT Internship (Cohort 02), Phase 1
Task	4. Common Web Vulnerabilities - Path Traversal
Platform	PortSwigger Web Security Academy
Lab	File path traversal, simple case (Apprentice)
Link	portswigger.net/web-security/file-path-traversal/lab-simple
Focus	path traversal, filename parameter, reading /etc/passwd
Status	Solved
Date	October 2026

1. Objective

Solve the PortSwigger lab "File path traversal, simple case": exploit a path traversal vulnerability in the product image feature to read the /etc/passwd file.

2. Concept / Understanding

- **Path traversal (directory traversal):** a vulnerability where user input (like a filename parameter) is used directly to read a file on the server, without checking it. This lets an attacker read files outside the folder the application intended.
- **../** means "go up one folder". Repeating it (../../../) climbs out of the web application's folder and reaches the root of the file system, from where any file path can be reached.
- In this lab, the image is loaded with a filename parameter, for example `/image?filename=1.jpg`. If the server just opens that path without checking it, changing the value can make it open a different file.
- **/etc/passwd** is a system file on Linux that lists the system's user accounts. It is commonly used in labs as a safe way to prove file read access, because it exists on almost every Linux server and is easy to recognize.

3. Practical Work

Lab: File path traversal, simple case. I did the following steps:

#	Step	What I did / found
1	Browsed a product page and let Burp Proxy capture the image request, then sent it to Repeater.	Request: GET /image?filename=1.jpg. The Render tab showed the normal product image.
2	Sent the request again as-is in Repeater, to check the baseline response.	Response: 200 OK, Content-Type image/jpeg, with the raw image bytes.
3	Changed the filename value to `../../../etc/passwd` and sent the request.	Response: 200 OK. The body now contained the text of the /etc/passwd file (a list of system users like root, daemon, bin, carlos, etc), not an image.

This confirmed the application does not validate the filename parameter, so it was possible to read a file outside the intended image folder. The lab was marked Solved.

4. Tools Used

- Burp Suite (Proxy and Repeater)

- Web browser
- PortSwigger Web Security Academy lab

5. Evidence



Figure 1: the lab page, objective is to retrieve /etc/passwd. Status: Solved.

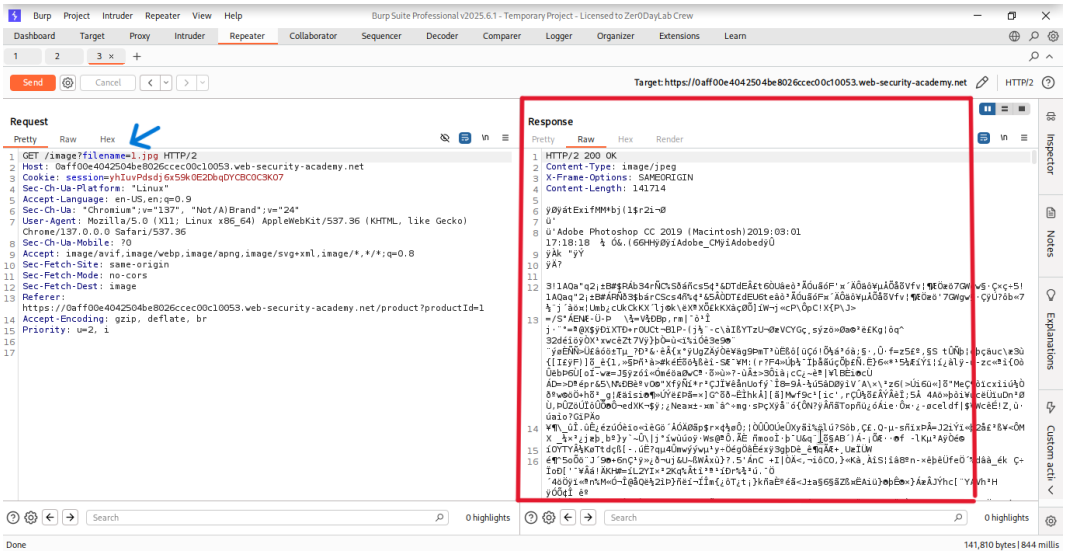


Figure 2: the normal request GET /image?filename=1.jpg, before any modification.

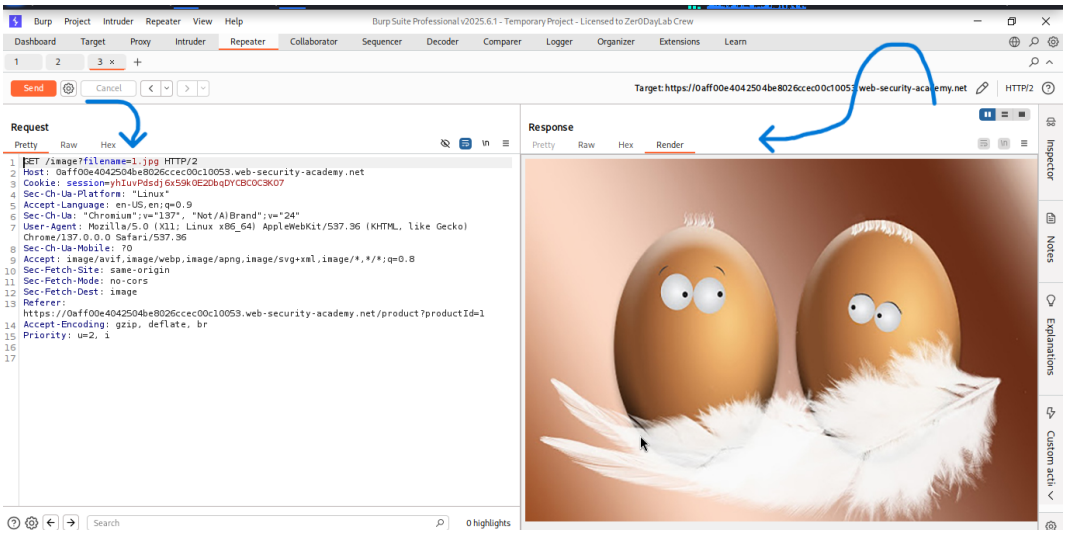


Figure 3: the Render tab showing the normal product image returned for filename=1.jpg.

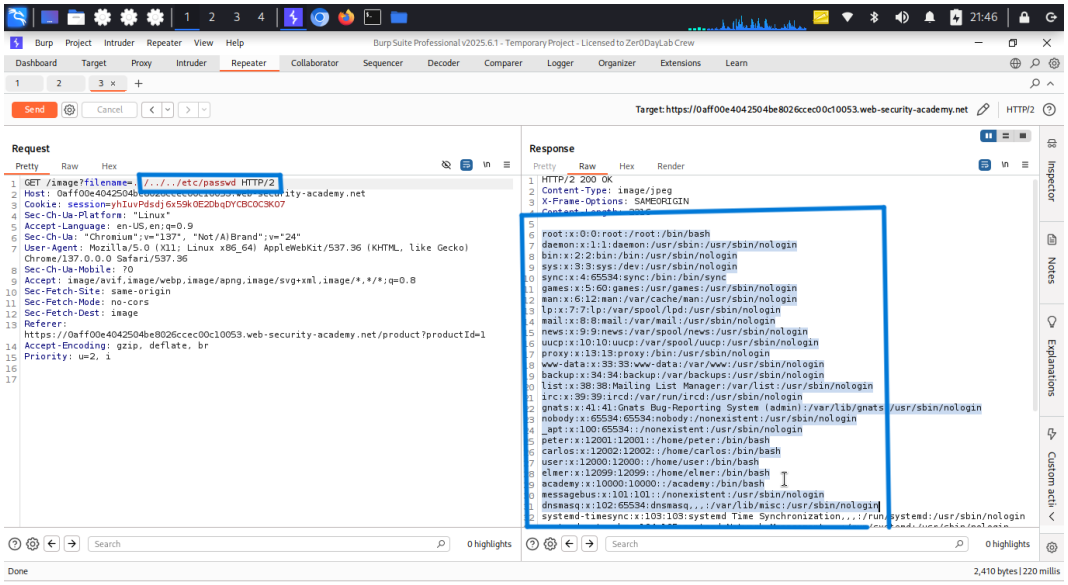


Figure 4: the modified request with filename=../../../etc/passwd. The response body shows the contents of /etc/passwd (the system's user accounts), proving the file read.

6. Result

Solved the lab. By changing the filename parameter to ../../../etc/passwd, I read the contents of the system's /etc/passwd file through the image loading feature.

7. Key Learnings

- A filename or file path coming from the user should never be trusted and used directly to open a file.
- ../ sequences let an attacker move up out of the intended folder and reach other files on the server.
- A simple test (loading a known file like /etc/passwd) can confirm the vulnerability before trying more sensitive files.
- Burp Repeater makes it easy to take one real request and test a changed parameter safely and repeatedly.

8. Impact and Remediation

Impact: an attacker can read any file the application's process has access to: configuration files, source code, credentials, or system files. This can expose secrets and lead to further compromise of the application or server.

Remediation:

- Avoid building file paths from user input. Where possible, use an ID that maps to a fixed, pre-approved file instead of a filename.
- If a filename must be accepted, validate it against a strict whitelist (allowed characters, no ../, no absolute paths).
- Resolve the final path and check that it still stays inside the intended base folder before opening the file.